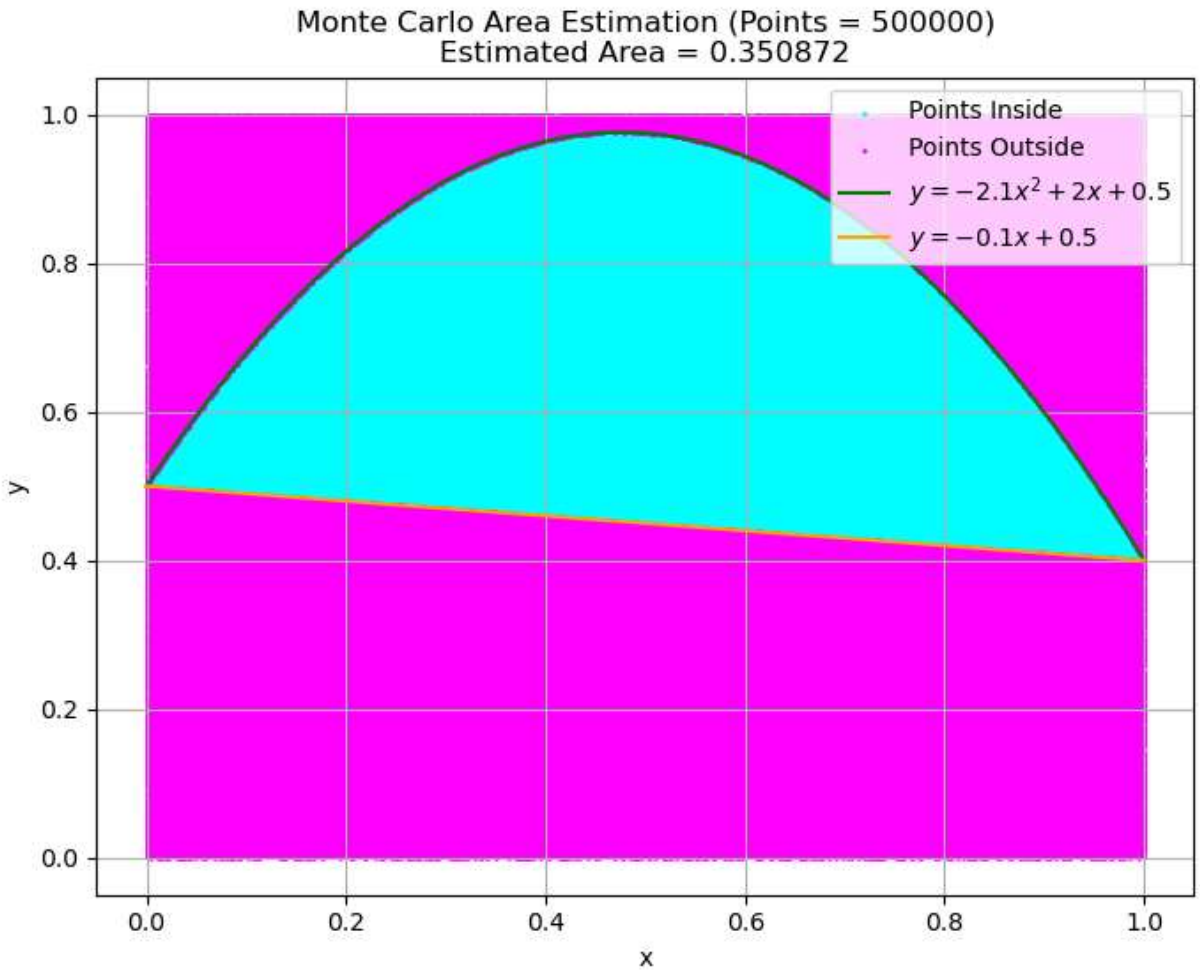
In [1]:
```
# Q4
```

In [2]:
```
import random
import matplotlib.pyplot as plt

def upper_curve(x):
    return -2.1 * x**2 + 2 * x + 0.5

def lower_curve(x):
    return -0.1 * x + 0.5

def estimate_area_monte_carlo(total_points):

    count_inside = 0
    x_inside = []
    y_inside = []
    x_outside = []
    y_outside = []

    x_start, x_end = 0, 1
    y_start, y_end = 0, 1
    bounding_area = (x_end - x_start) * (y_end - y_start)

    for _ in range(total_points):
        rand_x = random.uniform(x_start, x_end)
        rand_y = random.uniform(y_start, y_end)

        if lower_curve(rand_x) <= rand_y <= upper_curve(rand_x):
            count_inside += 1
            x_inside.append(rand_x)
            y_inside.append(rand_y)
        else:
            x_outside.append(rand_x)
            y_outside.append(rand_y)

    area_estimate = bounding_area * (count_inside / total_points)

    plt.figure(figsize=(8, 6))

    plt.scatter(x_inside, y_inside, color='cyan', s=1, label='Points Inside')
    plt.scatter(x_outside, y_outside, color='magenta', s=1, label='Points Outside')

    x_vals = [i / 100 for i in range(101)]
    plt.plot(x_vals, [upper_curve(x) for x in x_vals], color='green', label='$y = -
    plt.plot(x_vals, [lower_curve(x) for x in x_vals], color='orange', label='$y =

    plt.title(f'Monte Carlo Area Estimation (Points = {total_points})\nEstimated Ar
    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.grid(True)
    plt.show()

    return area_estimate
```

```
if __name__ == '__main__':
    total_points = 500000
    area = estimate_area_monte_carlo(total_points)

    print(f"The estimated area of the region is: {area:.6f}")
```

```
D:\anaconda\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Creating
legend with loc="best" can be slow with large amounts of data.
  fig.canvas.print_figure(bytes_io, **kw)
```




```
The estimated area of the region is: 0.350872
```

In [ ]: